HIGH-PERFORMANCE CUDA DATA AUGMENTATION PIPELINE

Authors:
Abhinav Singh Bhagtana
Vaibhav Shukla
Adi Ramanarayan

Affiliations:
Department of Computer Science

Project Guide:
Dr. Sandhya Parasnath Dubey

April 14, 2026

ABSTRACT

This project presents the design and implementation of a high-performance, GPU-accelerated data augmentation pipeline tailored for machine learning datasets. Traditional CPU-bound image processing introduces severe data-loading bottlenecks due to sequential execution paradigms. To resolve this, we leverage NVIDIA's Compute Unified Device Architecture (CUDA) to offload spatial and geometric transformations specifically Box Blur, Laplacian Sharpening, Rotation, Scaling, and Nearest-Neighbor Resizing—directly to the GPU.

By treating the augmentation sequence as a 5-bit binary integer, the system utilizes a bitmask-driven power set generator to programmatically apply 32 distinct combinations per input image. Furthermore, the architecture implements a dual-memory Ping-Pong buffer system to guarantee thread safety and eliminate redundant VRAM allocation overhead during the iterative processing loop. The resulting pipeline successfully generates complete, uncorrupted dataset variations at significantly accelerated speeds compared to host-bound execution, demonstrating the efficacy of parallel compute density in computer vision preprocessing tasks.

Chapter 1

INTRODUCTION

1.1 GENERAL INTRODUCTION TO THE TOPIC

In the modern landscape of artificial intelligence, Convolutional Neural Networks (CNNs) and other deep learning architectures achieve state-of-the-art performance primarily through the ingestion of massive datasets. However, acquiring and labeling millions of unique images is often prohibitively expensive or practically impossible. Data augmentation resolves this by synthetically expanding a dataset through geometric and photometric transformations—such as blurring, sharpening, scaling, and rotating.

While mathematically simple, these pixel-level transformations become computationally severe when applied sequentially across thousands of high-resolution images. Traditional Machine Learning pipelines often rely on the Central Processing Unit (CPU) for these tasks, resulting in severe data-loading bottlenecks that leave the Graphical Processing Unit (GPU) idle. This project proposes a high-performance solution by shifting the entire augmentation pipeline directly to the GPU utilizing NVIDIA's Compute Unified Device Architecture (CUDA), enabling massive parallelization and significantly reducing preprocessing latency.

1.2 ORGANIZATION

The remainder of this report is organized as follows: Chapter 2 formally defines the computational bottlenecks inherent in traditional data pipelines. Chapter 3 outlines the specific technical objectives of the proposed GPU-accelerated solution. Chapter 4 provides the foundational background on parallel compute paradigms and memory management. Chapter 5 details the core methodology, including the bitmask generator and the mathematics of affine transformations. Chapter 6 breaks down the C++ implementation details, specifically focusing on the Ping-Pong buffer architecture. Finally, Chapter 7 concludes the report.

1.3 AREA OF COMPUTER SCIENCE

This project fundamentally resides at the intersection of **High-Performance Computing (HPC)** and **Computer Vision**. It deals heavily with parallel compute algorithms, memory bandwidth optimization, and spatial image processing.

1.4 HARDWARE AND SOFTWARE REQUIREMENTS

The execution and compilation of this project require the following environment:

- **Hardware:**
  - An NVIDIA Graphical Processing Unit (GPU) with CUDA support.
  - A multi-core Central Processing Unit (CPU) for host-side orchestration.
- **Software:**
  - **Operating System:** A Linux-based distribution (e.g., Ubuntu).
  - **Compiler:** The NVIDIA CUDA Compiler (`nvcc`) and a standard C++ compiler (e.g., GCC).
  - **Libraries:** The CUDA Toolkit and the `stb_image` / `stb_image_write` single-header C libraries for image decoding and encoding.

Chapter 2

PROBLEM DEFINITION

The fundamental problem addressed by this project is the severe computational bottleneck introduced by sequential data preprocessing in modern Machine Learning (ML) pipelines.

As deep learning models grow in complexity, their capacity to ingest and process data often outpaces the system's ability to supply it. In a standard ML training loop, data augmentation is performed dynamically on the host CPU before the images are batched and transferred to the GPU for model training. Because CPU architectures are optimized for low-latency, sequential instruction execution across a few powerful cores, they are fundamentally ill-equipped to perform pixel-by-pixel mathematical transformations on high-resolution image matrices.

This paradigm creates a phenomenon known as "GPU Starvation." The highly parallelized training model sits idle, wasting computational cycles and power, while waiting for the CPU to finish applying sequential blurring, rotating, or scaling algorithms to the next batch of images.

Furthermore, naive implementations of sequential augmentation pipelines often suffer from significant memory overhead. Applying multiple distinct transformations (e.g., rotating, then sharpening, then scaling) typically requires allocating a new memory buffer for each intermediate step to prevent race conditions and data corruption. This continuous allocation and deallocation of memory further exacerbates the CPU bottleneck.

Therefore, the problem is twofold: to eliminate the sequential processing bottleneck by offloading the mathematical transformations to a massively parallelized architecture, and to engineer a memory-efficient pipeline that eliminates redundant memory allocation overhead during multi-step augmentations.

Chapter 3

OBJECTIVES

The primary objective of this project is to design, implement, and validate a high-performance, GPU-accelerated data augmentation pipeline capable of alleviating the host-side processing bottlenecks found in traditional Machine Learning workflows.

To achieve this primary goal, the project targets the following specific technical milestones:

- **GPU Offloading:** To implement spatial domain filters (Box Blur, Laplacian Sharpening) and affine transformations (Rotation, Scaling, Resizing) as heavily parallelized CUDA device kernels, shifting the computational burden entirely away from the host CPU.
- **Combinatorial Generation:** To design a mathematically robust bitmask generator capable of treating the sequence of available operations as an N-bit integer, automatically generating and executing all 2^N (32) possible transformation combinations without redundant code branching.
- **Memory Optimization:** To engineer a Ping-Pong buffer architecture in VRAM that guarantees thread safety and prevents race conditions without relying on latency-heavy, iterative `cudaMalloc` allocations during the augmentation loop.
- **Dynamic Canvas Handling:** To implement dynamic state-tracking logic that allows the system to seamlessly handle on-the-fly resolution changes (e.g., nearest-neighbor upscaling) within the augmentation sequence without triggering out-of-bounds memory accesses.
- **Dataset Readiness:** To ensure the final output is immediately deployable for deep learning models by automatically generating a perfectly mapped `manifest.csv` file alongside the uncorrupted output image directory.

Chapter 4

BACKGROUND

To understand the architectural decisions made in this pipeline, it is necessary to establish the foundational concepts of dataset synthesis, parallel compute paradigms, and digital memory representation.

4.1 THE NECESSITY OF DATA AUGMENTATION

Deep learning models, particularly Convolutional Neural Networks (CNNs), require massive amounts of labeled training data to generalize effectively to real-world scenarios. When trained on limited datasets, models are prone to overfitting—memorizing the training data rather than learning underlying patterns. Data augmentation solves this by applying label-preserving geometric and photometric transformations to existing images, artificially expanding the dataset's variance and scale without the high cost of manual data collection.[1]

4.2 SEQUENTIAL VS. PARALLEL COMPUTE ARCHITECTURES

Traditional data loading pipelines often rely on the Central Processing Unit (CPU) to apply augmentations. However, CPU architecture is optimized for low-latency sequential task execution. Processing a high-resolution image requires iterating through millions of pixels sequentially, creating a massive data-starvation bottleneck that forces the Machine Learning model to sit idle while waiting for the next batch of images.

Graphics Processing Units (GPUs) solve this via a Single Instruction, Multiple Threads (SIMT) architecture. Using NVIDIA's Compute Unified Device Architecture (CUDA)[2], developers can offload highly parallelizable workloads to the GPU. Instead of one CPU core processing millions of pixels sequentially, a GPU can spin up thousands of lightweight threads simultaneously, mapping one thread to each individual pixel to process the entire image in a fraction of a millisecond.

4.3 DIGITAL IMAGE REPRESENTATION

While humans perceive digital images as 2D grids of color, the hardware stores them as flattened, one-dimensional contiguous arrays of memory. An image with a specific width (W), height (H), and number of color channels (C) is stored as an array of length $W \times H \times C$.

For standard RGB images ($C = 3$), the pixel values are interleaved (Red, Green, Blue, Red, Green, Blue...). Therefore, to perform mathematical operations on a specific pixel at a 2D coordinate (x, y), the exact 1D memory index must be algebraically calculated as:

$$Index = (y \times W + x) \times C \tag{4.1}$$

Understanding this flattened memory stride is critical for designing device kernels that can accurately map 2D spatial transformations without triggering memory segmentation faults.

Chapter 5

METHODOLOGY

The core methodology of this project relies on combining foundational computer vision mathematics with highly parallelized GPU execution. The augmentation pipeline applies three distinct classes of operations: Convolutional Filtering, Affine Transformations, and Spatial Interpolation.

5.1 COMBINATORIAL GENERATION VIA BITMASKING

To generate a comprehensive dataset without writing redundant execution branches, the pipeline utilizes a bitmask-driven power set generator. Given N distinct augmentation kernels, the system can generate 2^N unique combinations. By treating the augmentation sequence as an N-bit binary integer (e.g., 00101_2 representing Blur and Scale), a single loop from 0 to $2^N - 1$ programmatically applies every possible permutation.

5.2 CONVOLUTIONAL FILTERS: BLUR AND SHARPEN

The spatial domain filters operate by convolving a 3×3 kernel matrix over the input image array.[1]

For the Box Blur operation, a uniform averaging kernel is applied to simulate optical defocus by calculating the mean of a pixel's 8 immediate neighbors. The normalized matrix is defined as:

$$K_{blur} = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \tag{5.1}$$

Applying the Blur Kernel on the original image yields a slightly blurred version of the same image as shown below.

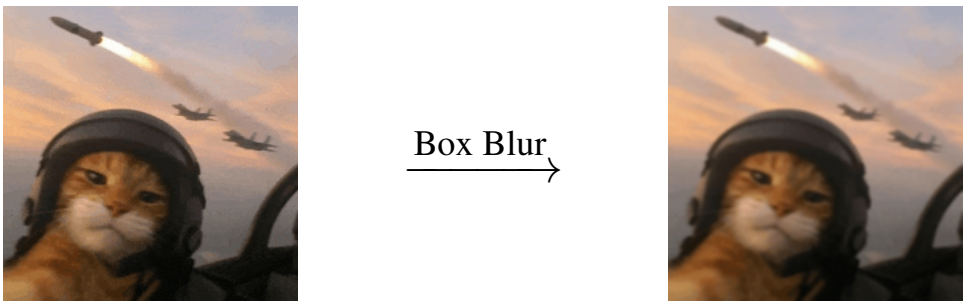

Fig. 5.1 Visual demonstration of the Blur Kernel.

The Box Blur kernel acts as a low-pass spatial filter that mathematically averages neighboring pixel intensities. By intentionally degrading the high-frequency details of the source image, this transformation simulates real-world focal variance, motion blur, and poor sensor quality. In the context of machine learning, introducing blurred data is highly beneficial because it prevents the convolutional neural network from overfitting to hyper-specific, localized textures. Instead, it forces the model to generalize and identify the underlying macro-structures and geometric shapes of the subject matter, leading to a significantly more robust predictive algorithm.

For Sharpening, a Laplacian filter is utilized to extract and amplify high-frequency edge data. The Laplacian filter matrix is defined as:

$$K_{laplacian} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} \tag{5.2}$$

Applying the Sharpening Kernel on the original image yields a slightly sharpened version of the same image as shown below.

Fig. 5.2 Visual demonstration of the Sharpening Kernel.

the Laplacian Sharpening kernel operates as a high-pass spatial filter designed to artificially amplify local contrast. By calculating the second-order unmixed derivatives of the pixel grid, this kernel aggressively highlights edge boundaries and generates localized overshoot (halo effects) at the intersection of dark and light pixels. For machine learning pipelines, injecting artificially sharpened images into the training dataset prepares the neural network to handle over-exposed source data and harsh lighting variations. This trains the model to extract and prioritize structural edge data even when processing images with severe, localized contrast spikes.

To prevent integer overflow and color corruption artifacts (wrap-around) during execution, the resulting scalar value for each pixel is strictly clamped to the standard 8-bit RGB range of $[0, 255]$ before being written to device memory.

5.3 AFFINE TRANSFORMATIONS: ROTATION AND SCALING

To prevent "hole" artifacts—where destination pixels are left uncalculated due to floating-point rounding—the pipeline utilizes Inverse Mapping for geometric transformations. Instead of projecting a source pixel forward, each thread calculates its destination coordinate and maps backward to the nearest source pixel.[1].

For the Rotation kernel, the inverse mapping algorithm applies the 2D rotation matrix around the image center (c_x, c_y) by an angle θ. The source coordinates (x_{src}, y_{src}) for a given destination thread (x_{dst}, y_{dst}) are calculated as:

$$x_{src} = (x_{dst} - c_x)\cos(\theta) + (y_{dst} - c_y)\sin(\theta) + c_x \tag{5.3}$$

$$y_{src} = -(x_{dst} - c_x)\sin(\theta) + (y_{dst} - c_y)\cos(\theta) + c_y \tag{5.4}$$

The application of the rotation kernel with $\theta = 45^\circ$ results in an augmented image rotated by 45° clockwise.

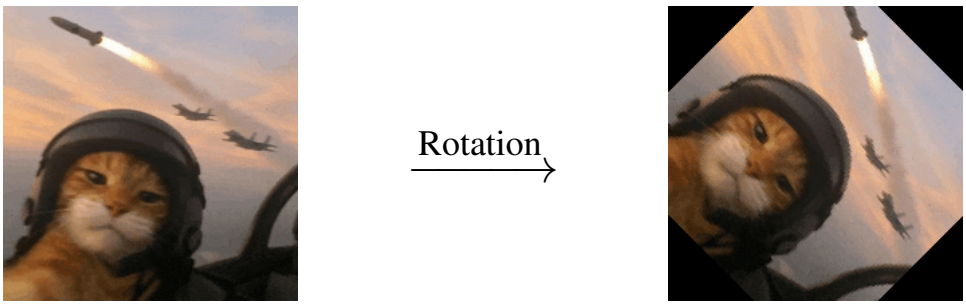

Fig. 5.3 Visual demonstration of the Rotation Kernel.

During rotation, regions of the original image naturally extend beyond the bounding canvas, resulting in cropped borders and zero-padded (black) regions. This effect is intentional and highly beneficial for machine learning. By introducing zero-padded borders and shifting the spatial framing, the pipeline diversifies the dataset and trains the target ML model to achieve rotational invariance, preventing it from overfitting to standard image orientations.

Similarly, the Scale kernel performs a focal zoom by an isotropic scale factor S. To ensure the zoom remains centered rather than stretching unevenly from the top-left origin $(0,0)$, the inverse mapping algorithm is defined as:

$$x_{src} = \frac{x_{dst} - c_x}{S} + c_x \tag{5.5}$$

$$y_{src} = \frac{y_{dst} - c_y}{S} + c_y \tag{5.6}$$

where S represents the scalar zoom multiplier (e.g., $S = 1.5$ corresponds to a 150% magnification of the central focal area).

Applying the scaling kernel with a transformation factor of $S = 1.5$ results in a 150% magnification of the original pixel grid, preserving the image dimensions.

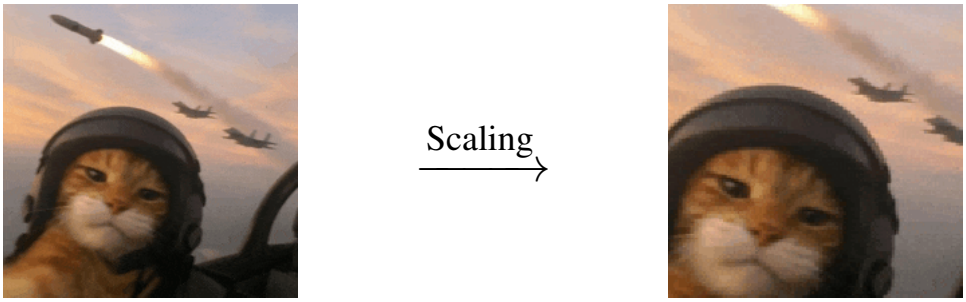

Fig. 5.4 Visual demonstration of the Scale Kernel.

Implementing this spatial resizing is critical for developing a comprehensive machine learning dataset, as it artificially simulates varying camera-to-subject distances. By feeding the neural network multiple scaled variations of the same source image, the model is trained to achieve strict scale invariance. This ensures the algorithm learns to identify the core geometric features and relative proportions of an object, rather than overfitting to its absolute pixel footprint on the canvas. Consequently, the final trained CNN maintains high predictive accuracy regardless of whether the target subject appears as a massive close-up or a distant, low-resolution element within the frame.

5.4 DIMENSIONAL RESIZING: NEAREST-NEIGHBOR INTERPOLATION

Unlike the scaling transformation—which applies an affine zoom while maintaining a static canvas size—the resizing operation fundamentally alters the memory footprint and spatial dimensions of the dataset. To map the original source image of dimensions (W_{src}, H_{src}) to a new target resolution (W_{dst}, H_{dst}), the pipeline utilizes Nearest-Neighbor Interpolation.[1]

Nearest-neighbor is specifically chosen for this high-performance pipeline due to its low computational overhead and its ability to perfectly preserve original pixel intensities without introducing interpolated blurring.

To map the pixels correctly, the dimensional scaling ratios must first be calculated:

$$R_x = \frac{W_{src}}{W_{dst}} \tag{5.7}$$

$$R_y = \frac{H_{src}}{H_{dst}} \tag{5.8}$$

For any given GPU thread mapped to a destination coordinate (x_{dst}, y_{dst}), the algorithm calculates the discrete coordinate of the nearest original source pixel (x_{src}, y_{src}) using

the mathematical floor function:

$$x_{src} = \lfloor x_{dst} \times R_x \rfloor \tag{5.9}$$

$$y_{src} = \lfloor y_{dst} \times R_y \rfloor \tag{5.10}$$

Application of the spatial resizing kernel maps the 220×220 source image to an augmented target buffer of 500×500 pixels. This expansion yields uniform scaling coefficients of $Rx = Ry \approx 2.27$, resulting in a final image area that is approximately $5.16\times$ larger than the original coordinate space.

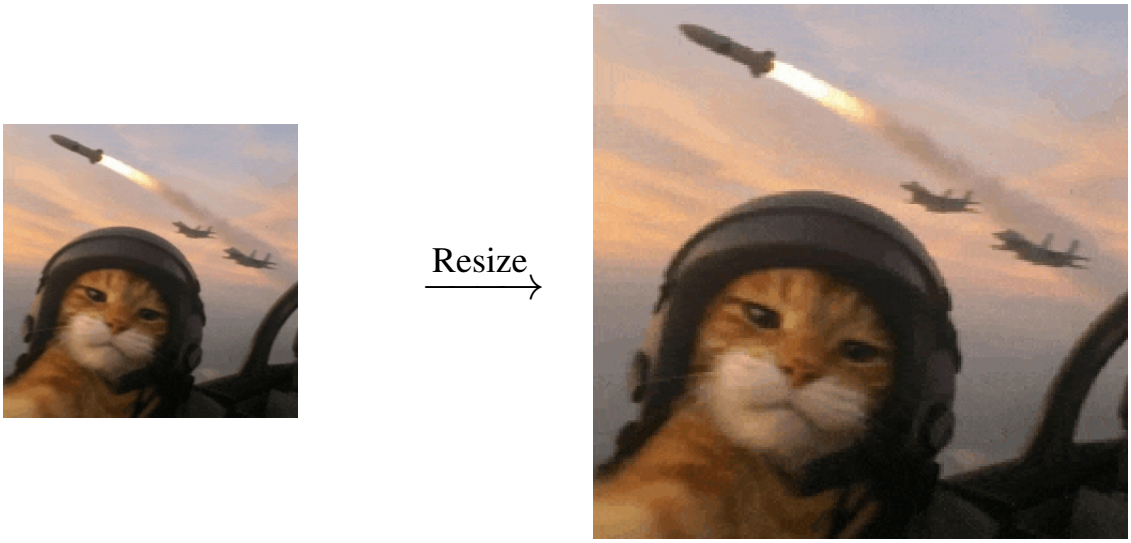

Fig. 5.5 Visual demonstration of the Resize Kernel.

Deploying this resizing kernel is critical for developing a comprehensive machine learning dataset, as it artificially simulates data captured across varying camera resolutions. By projecting source images into an expanded spatial grid, the model is trained to achieve strict resolution invariance. This ensures the algorithm learns to extract geometric features and relative proportions without relying on a highly dense, native pixel footprint. Consequently, the final trained CNN maintains high predictive accuracy regardless of whether it processes a low-resolution native capture or a mathematically upscaled image.

Chapter 6

IMPLEMENTATION DETAILS

The system architecture is strictly divided between Host (CPU) orchestration and Device (GPU) execution to maximize memory bandwidth efficiency and parallel compute density.

6.1 HOST-DEVICE SEPARATION AND MEMORY MANAGEMENT

The standard C entry point (`main.cu`) handles all file I/O utilizing the `stb_image` single-header library[3]. Once the raw pixel array is decoded, it is transferred across the PCIe bus via `cudaMemcpy`. The actual mathematical execution is isolated in a separate `kernels.cu` file, preventing CUDA-specific syntax from polluting the high-level orchestration logic.

6.2 THE PING-PONG BUFFER ARCHITECTURE

A naive sequential pipeline requires allocating new device memory for every augmentation step, which introduces massive latency due to `cudaMalloc` overhead. Furthermore, executing in-place modifications (reading and writing to the same array simultaneously) triggers race conditions across concurrent CUDA threads, resulting in visual tearing and ghosting artifacts.

To solve this, the system implements a Ping-Pong Buffer architecture. Exactly two maximum-capacity buffers (Buffer A and Buffer B) are allocated in VRAM prior to execution. The active device pointer array (`d_src`) is designated strictly as read-only, while the destination pointer (`d_dst`) is write-only. After each kernel execution, the CPU swaps the memory pointers. This completely eliminates memory reallocation inside the loop while guaranteeing thread safety[2].

6.3 DYNAMIC STATE TRACKING

Because the Resizing kernel dynamically alters the dimensional footprint of the active image, the pipeline features Dynamic State Tracking. VRAM is pre-allocated based on the absolute maximum possible dimensions (e.g., 500×500). Integer state variables (`current_w` and `current_h`) are updated mid-loop exclusively when the Resize bitmask is triggered. This ensures subsequent kernels calculate their grid/block dimensions accurately without accessing out-of-bounds memory.

Chapter 7

CONTRIBUTION SUMMARY

This project was a highly collaborative effort, with computational and architectural responsibilities explicitly divided to ensure efficient parallel development and debugging. The specific technical contributions of each team member are outlined below:

- **Vaibhav and Adi:** Led the development of the device-side execution logic. They were strictly responsible for designing and implementing the CUDA kernel functions within `kernels.cu`. This included the mathematical derivation and coding of the spatial domain filters (Box Blur, Laplacian Sharpening) as well as the inverse mapping algorithms required for the affine transformations (Rotation, Scaling, Resizing).
- **Abhinav:** Led the development of the host-side orchestration and overall pipeline architecture. He was responsible for the main logic flow within `main.cu`, which involved implementing the bitmask combinatorial generator, engineering the Ping-Pong buffer memory management system to eliminate race conditions, and handling all image array decoding/encoding utilizing the `stb_image` libraries.

REFERENCES

[1] R. Szeliski, *Computer Vision: Algorithms and Applications*. Springer Science & Business Media, 2010.

[2] NVIDIA Corporation, "Cuda c++ programming guide," 2024. Accessed: 2024.

[3] S. Barrett, "stb_image: Single-file public domain library for c/c++," 2024. Accessed: 2024.